

CS 3563: Introduction to DBMS II

Homework 3-4: Query Processing and Optimization

Name: Pathri Vidya Praveen
Roll No: CS24BTECH11047

March 10, 2026

Question 1

(a) Block Access Estimation for Join Strategies

Initial Parameters:

- **Buffers (M):** 3
- **Relation R:** $n_R = 20,000$ tuples, $blockingFactor_R = 25$ tuples/block $\implies b_R = \lceil 20,000/25 \rceil = 800$ blocks.
- **Relation S:** $n_S = 45,000$ tuples, $blockingFactor_S = 30$ tuples/block $\implies b_S = \lceil 45,000/30 \rceil = 1500$ blocks.

(i) Tuple-based Nested Loop Join

We use the smaller relation (R) as the outer loop. For every tuple in R, the entire inner relation S must be scanned.

- **Formula:** $Cost = b_R + (n_R \times b_S)$
- **Calculation:** $800 + (20,000 \times 1500)$
- **Total Block Accesses:** 30,000,800

(ii) Block Nested Loop Join

With $M = 3$, we allocate 1 buffer for reading R (outer), 1 for S (inner), and 1 for output. For each block of R, we scan all blocks of S.

- **Formula:** $Cost = b_R + (b_R \times b_S)$
- **Calculation:** $800 + (800 \times 1500)$
- **Total Block Accesses:** 1,200,800

(iii) Sort-Merge Join

Both unsorted relations must be sorted using external merge sort ($M = 3$), followed by a single merge pass.

- **Sort R:**
 - Initial runs: $\lceil 800/3 \rceil = 267$.
 - Total passes: $1 + \lceil \log_2(267) \rceil = 10$.
 - Cost: $2 \times b_R \times \text{passes} = 2 \times 800 \times 10 = 16,000$.
- **Sort S:**
 - Initial runs: $\lceil 1500/3 \rceil = 500$.
 - Total passes: $1 + \lceil \log_2(500) \rceil = 10$.
 - Cost: $2 \times b_S \times \text{passes} = 2 \times 1500 \times 10 = 30,000$.
- **Merge Phase:** Read both sorted relations exactly once.
 - Cost: $b_R + b_S = 800 + 1500 = 2300$.
- **Total Block Accesses:** $16,000 + 30,000 + 2300 = \mathbf{48,300}$

(iv) Index Join

We assume a B^+ -tree index exists on relation R (inner) and S is the outer relation. Each tuple of S requires descending the index and accessing the corresponding tuple in R .

- **Height of the B^+ -tree (h):**
 - **Worst Case (Half-full nodes):** With a pointer capacity of 20, the minimum fan-out is $f = 10$. $h_{worst} = \lceil \log_{10}(20,000) \rceil = 5$.
 - **Best Case (Full nodes):** With maximum capacity, the fan-out is $f = 20$. $h_{best} = \lceil \log_{20}(20,000) \rceil = 4$.
- **Cost per probe:** h blocks for index traversal + 1 block for data fetch.
 - **Worst Case:** $5 + 1 = 6$ accesses.
 - **Best Case:** $4 + 1 = 5$ accesses.
- **Total Block Accesses Calculation:** Cost = $b_S + (n_S \times \text{cost per probe})$.
 - **Worst Case:** $1500 + (45,000 \times 6) = 271,500$.
 - **Best Case:** $1500 + (45,000 \times 5) = 226,500$.
- **Final Result:** Total Cost =
$$\begin{cases} 271,500 & \text{(Worst Case)} \\ 226,500 & \text{(Best Case)} \end{cases}$$

(v) Hash Join

Since the hash partitions fit entirely in memory, the algorithm requires only a single partition phase and a single probe phase.

- **Formula:** $\text{Cost} = 3 \times (b_R + b_S)$
- **Calculation:** $3 \times (800 + 1500)$
- **Total Block Accesses:** **6,900**

(b) Computing Left Outer Joins

All three standard algorithms can compute a left outer join by modifying how unmatched tuples from the left relation (R) are handled:

1. **Nested Loops Join:** R is the outer loop. A boolean flag tracks matches for each outer tuple. If the inner scan of S completes without setting the flag, the tuple from R is output padded with NULL values.
2. **Sort-Merge Join:** During the merge phase, if the pointer for R points to a key strictly smaller than the current key in S (indicating no match exists), the tuple from R is output padded with NULLs, and the pointer for R advances.
3. **Hash Join:** S is used as the build relation to create the hash table. R is streamed during the probe phase. If a tuple from R probes the hash table and finds no match, it is immediately output padded with NULLs.

Question 2

(a) Estimated Result Cardinality

The System R optimizer calculates the expected number of tuples returned by a query block (*QCARD*) by multiplying the total cardinality of the relation (*NCARD*) by the selectivity factor (*F*) of each boolean factor in the **WHERE** clause.

1. **Salary > 40000:** For an arithmetic range comparison, the factor is derived using linear interpolation:

$$F_1 = \frac{\text{HighKey} - \text{Value}}{\text{HighKey} - \text{LowKey}} = \frac{100,000 - 40,000}{100,000 - 20,000} = \frac{60,000}{80,000} = 0.75$$

2. **Age = 45:** For an equality predicate on an indexed column, the factor is the reciprocal of the number of distinct keys:

$$F_2 = \frac{1}{\text{ICARD}(I_2)} = \frac{1}{40} = 0.025$$

3. **CSY = RSY:** For a column-to-column comparison where one column is indexed (I_3), the selectivity is treated as $1/\text{ICARD}$ of the indexed column:

$$F_3 = \frac{1}{\text{ICARD}(I_3)} = \frac{1}{25} = 0.04$$

Final Calculation:

$$\text{Estimated Cardinality} = NCARD \times F_1 \times F_2 \times F_3$$

$$\text{Estimated Cardinality} = 100,000 \times 0.75 \times 0.025 \times 0.04 = \mathbf{75} \text{ tuples}$$

(b) Access Path Costs

The total access cost in System R is a weighted measure of I/O (page fetches) and CPU (RSI calls). We consider only I/O costs (weight $W = 0$).

1. Segment Scan

The cost of a segment scan is the total number of non-empty pages in the segment:

$$\text{Cost} = \frac{TCARD}{P} = \frac{1000}{0.8} = \mathbf{1250} \text{ pages}$$

2. Salary Index Scan (I_1)

Since the table is physically sorted on Salary, I_1 is a **clustered index**.

- **Formula:** $F(\text{preds}) \times (NINDEX(I_1) + TCARD)$
- **Calculation:** $0.75 \times (500 + 1000) = 0.75 \times 1500 = \mathbf{1125}$ pages

3. Age Index Scan (I_2)

I_2 is an **unclustered index**. Because the expected data pages ($\frac{TCARD}{ICARD(I_2)} = 25$) fit within the 1200-page buffer pool, we use the buffered cost formula:

- **Formula:** $F(\text{preds}) \times (NINDEX(I_2) + TCARD)$
- **Calculation:** $0.025 \times (20 + 1000) = 0.025 \times 1020 = \mathbf{25.5}$

4. RSY Index Scan (I_3)

I_3 is also an unclustered index. With a large buffer, the matching cost would be $0.04 \times (10 + 1000) = 40.4$. However, if the index is not used as a matching path for filtering, System R evaluates it as a full index scan:

- **Formula:** $NINDEX(I_3) + TCARD$
- **Calculation:** $10 + 1000 = \mathbf{1010}$ pages

Summary Table

Access Path	Type	Formula (I/O only)	Cost
Segment Scan	N/A	$TCARD/P$	1250
Salary Index (I_1)	Clustered	$F_1 \times (NINDEX + TCARD)$	1125
Age Index (I_2)	Unclustered	$F_2 \times (NINDEX + TCARD)$ (Buffered)	25.5
RSY Index (I_3)	Unclustered	$NINDEX + TCARD$ (Non-matching)	1010

Optimal Plan: The **Age Index Scan (I_2)** is chosen because it minimizes the total I/O cost at 25.5 pages

Question 3

For the relation schema $R = \{A, C, D, G, L, P, R, S, T, X\}$, we are given the following set of functional dependencies F :

$$F = \{S \rightarrow ALX, C \rightarrow P, DRT \rightarrow C, DPT \rightarrow R, DST \rightarrow R, CS \rightarrow G, R \rightarrow T, L \rightarrow A, GR \rightarrow DS\}$$

We must evaluate the assignment partner's claim that the decomposition into $\{CP, RT, AL, CDR, LSX, DGRS\}$ is a lossless-join, BCNF, and dependency-preserving decomposition.

1. Finding the Candidate Key

First, we observe the functional dependencies to find any attributes that do not appear on the right-hand side (RHS) of any FD. However, all attributes, including D and G , appear on the RHS of at least one FD ($GR \rightarrow DS$ and $CS \rightarrow G$).

Let us compute the closure of $\{G, R\}$ to check if it is a candidate key:

$$\begin{aligned} \{G, R\}^+ &= \{G, R\} \\ \text{Apply } R \rightarrow T &\implies \{G, R, T\} \\ \text{Apply } GR \rightarrow DS &\implies \{D, G, R, S, T\} \\ \text{Apply } S \rightarrow ALX &\implies \{A, D, G, L, R, S, T, X\} \\ \text{Apply } DRT \rightarrow C &\implies \{A, C, D, G, L, R, S, T, X\} \quad (\text{since we have } D, R, T) \\ \text{Apply } C \rightarrow P &\implies \{A, C, D, G, L, P, R, S, T, X\} \end{aligned}$$

Thus, $\{G, R\}^+ = R$. This means $\{G, R\}$ determines all attributes in the relation.

Next, we check for minimality:

- $\{G\}^+ = \{G\}$ (Cannot derive R or other attributes)
- $\{R\}^+ = \{R, T\}$ (Cannot derive G or other attributes)

Since no proper subset of $\{G, R\}$ can determine all attributes in R , it is minimal.

Candidate Key = $\{GR\}$

2. Dependency Preservation Check

A decomposition $R \rightarrow R_1, R_2, \dots, R_n$ is dependency-preserving if:

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

where F_i is the projection of F onto R_i .

Consider the functional dependency $CS \rightarrow G$. For this dependency to be preserved locally without performing a join, there must exist some relation R_i in the decomposition that contains both C and S .

Looking at the given decomposition:

Relation	Attributes
R_1	C, P
R_2	R, T
R_3	A, L
R_4	C, D, R
R_5	L, S, X
R_6	D, G, R, S

No single relation contains both C and S together. Therefore, $CS \rightarrow G$ cannot be checked without joining relations. Similarly, dependencies like $DRT \rightarrow C$ and $DST \rightarrow R$ span across multiple relations

Decomposition is NOT dependency preserving

3. BCNF Check

A relation R_i is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$ that holds in R_i , X is a superkey of the relation R_i .

Relation	Projected FD	Is LHS a superkey of the relation?	BCNF?
$R_1(C, P)$	$C \rightarrow P$	Yes ($\{C\}^+ \supseteq \{C, P\}$)	Yes
$R_2(R, T)$	$R \rightarrow T$	Yes ($\{R\}^+ \supseteq \{R, T\}$)	Yes
$R_3(A, L)$	$L \rightarrow A$	Yes ($\{L\}^+ \supseteq \{A, L\}$)	Yes
$R_4(C, D, R)$	$DR \rightarrow C^*$	Yes ($\{DR\}^+ \supseteq \{C, D, R\}$)	Yes
$R_5(L, S, X)$	$S \rightarrow LX$	Yes ($\{S\}^+ \supseteq \{L, S, X\}$)	Yes
$R_6(D, G, R, S)$	$GR \rightarrow DS$	Yes ($\{GR\}^+ \supseteq \{D, G, R, S\}$)	Yes

($DR \rightarrow C$ holds in R_4 because, in the global closure, $R \rightarrow T$ provides T , and $DRT \rightarrow C$ provides C .)

Every relation is in BCNF

4. Lossless-Join Check

A decomposition of R into R_i, R_j is lossless if:

$$(R_i \cap R_j) \rightarrow R_i \quad \text{or} \quad (R_i \cap R_j) \rightarrow R_j$$

We can systematically reconstruct R by joining the sub-relations, ensuring that at each step, the intersection is a candidate key for at least one of the relations being joined:

1. $R_1(C, P) \bowtie R_4(C, D, R)$: The intersection is $\{C\}$. Since $C \rightarrow P$, $\{C\}$ is a key for R_1 . This join is lossless and forms $R_{1,4}(C, D, P, R)$.
2. $R_2(R, T) \bowtie R_{1,4}(C, D, P, R)$: The intersection is $\{R\}$. Since $R \rightarrow T$, $\{R\}$ is a key for R_2 . This join is lossless and forms $R_{1,2,4}(C, D, P, R, T)$.
3. $R_{1,2,4}(C, D, P, R, T) \bowtie R_6(D, G, R, S)$: The intersection is $\{D, R\}$. Inside $R_{1,2,4}$, $\{D, R\}$ determines all attributes ($R \rightarrow T$, then $DRT \rightarrow C$, then $C \rightarrow P$). Since $\{D, R\}$ is a superkey for $R_{1,2,4}$, this join is lossless and forms $R_{1,2,4,6}(C, D, G, P, R, S, T)$.
4. $R_3(A, L) \bowtie R_{1,2,4,6}(C, D, G, P, R, S, T)$: The intersection is $\{L\}$. Since $L \rightarrow A$, $\{L\}$ is a key for R_3 . This join is lossless and forms $R_{3,5}(A, L, S, X)$.
5. $R_{1,2,4,6} \bowtie R_{3,5}$: The intersection between the two remaining clusters is $\{S\}$. Since $S \rightarrow ALX$ (which covers A, L, S, X), $\{S\}$ is a key for $R_{3,5}$. This final join is lossless and reconstructs the original schema R .

The decomposition is Lossless

Final Conclusion

Property	Status
Lossless Join	Yes
BCNF	Yes
Dependency Preserving	No

Therefore, the assignment partner's claim is incorrect. Although the decomposition is in BCNF and is a lossless join, it does not preserve all functional dependencies of the original schema F .